

Universidade Federal de Uberlândia
Faculdade de Computação

Engenharia de Software
Prof. Fabiano Azevedo Dorça

Testes Automatizados
(Parte 01)

Testes Automatizados

- A automação de testes envolve o uso de ferramentas e scripts para executar testes em software de forma automática, comparando os resultados esperados com os resultados reais.
- Isso é fundamental por várias razões:
- **1. Eficiência e Produtividade:**
 - Testes automatizados podem ser executados repetidamente com pouco esforço manual, permitindo que os desenvolvedores se concentrem em outras tarefas.

Exemplo: Um teste de regressão automatizado pode ser executado sempre que um novo código é integrado, garantindo que novas mudanças não quebrem funcionalidades existentes.

Testes Automatizados

- **2. Cobertura de Testes:**

- Automação permite a execução de um grande número de testes em um curto período, aumentando a cobertura de testes e a detecção de bugs.
- Exemplo: Scripts de teste automatizados podem validar milhares de cenários de uso em várias configurações de hardware e software.

- **3. Consistência e Precisão:**

- Testes automatizados são executados de forma consistente, eliminando a variabilidade introduzida por testes manuais.
- Exemplo: Testes de desempenho automatizados podem medir precisamente a velocidade e a capacidade de resposta de uma aplicação sob diferentes cargas.

Testes Automatizados

- **4. Feedback Rápido:**

- Com a automação, os desenvolvedores recebem feedback imediato sobre o estado do software, permitindo correções rápidas.
- Exemplo: Um pipeline de integração contínua (CI) que executa testes automatizados após cada commit fornece feedback instantâneo sobre a qualidade do código.

Testes Automatizados

- Pirâmide de Testes automatizados
 - define a quantidade e o tipo de testes que deveriam ser escritos para um projeto saudável:
 - **Base**: Testes de Unidade (muitos): São rápidos, baratos e testam pequenas partes do código isoladamente.
 - **Meio**: Testes de Integração e Serviço (quantidade média): Verificam a comunicação entre componentes ou serviços.
 - **Topo**: Testes de Interface do Usuário (UI) / End-to-End (E2E) (poucos): São lentos, caros e simulam o comportamento real do usuário final.
 - A ideia é ter muitos testes na base (rápidos) e poucos no topo (lentos), garantindo feedback rápido e confiável.

Testes Automatizados

- **1. Testes de Unidade (Unit Tests)**
 - Testam a menor parte testável do software, como uma função, método ou classe, isoladamente (em sandbox).
 - Dependências externas (bancos de dados, APIs, sistemas de arquivos) são simuladas usando Mocks ou Stubs.
- Objetivo: Validar a lógica interna de uma unidade de código. Garantir que cada "tijolo" do sistema funciona corretamente por si só.

Testes Automatizados

- **Ferramentas Principais:**

- JUnit: O padrão clássico para Java.
- NUnit: O equivalente ao JUnit para a plataforma .NET (C#).
- pytest: Framework moderno, simples e poderoso para Python. Muito popular.
- Jest: Framework all-in-one extremamente popular para JavaScript/Node.js. Inclui recursos de mocking e coverage por padrão.
- Mocha + Chai + Sinon: Combinação clássica e muito flexível para JavaScript. Mocha (test runner), Chai (assertions library), Sinon (para criar mocks e stubs).
- RSpec: Framework para testes em Ruby, seguindo o estilo BDD (Behavior-Driven Development).

Testes Automatizados

- 2. Testes de Integração (Integration Tests)
 - Testam a interação entre dois ou mais componentes ou módulos integrados.
 - O foco é na comunicação e no fluxo de dados entre eles (e.g., uma classe que se comunica com um banco de dados real, uma API que chama outra API).
- Objetivo: Encontrar defeitos nas interfaces e interações entre os módulos integrados.

Testes Automatizados

- **Ferramentas Principais:**
- JUnit / NUnit / pytest: Também são usados para orquestrar testes de integração.
- TestContainers: Ferramenta revolucionária que permite levantar containers Docker efêmeros (banco de dados, message brokers, etc.) para criar um ambiente de integração realista e descartável. Muito usado com Java, .NET, Python, Go, etc.
- RestAssured: Biblioteca Java para testar e validar APIs REST de forma simples e fluente.
- Postman / Newman: Postman (GUI) é excelente para criar e debuggar coleções de requests de API. Newman é o CLI do Postman, permitindo executar essas coleções em pipelines de CI/CD.

Testes Automatizados

- **3. Testes de Sistema / End-to-End (E2E)**
 - Testam o sistema completo e integrado, de ponta a ponta, simulando o comportamento de um usuário real.
 - Envolvem a interface do usuário, backend, banco de dados, rede, etc.
 - Objetivo: Validar se o fluxo completo de uma funcionalidade atende aos requisitos de negócio do início ao fim.

Testes Automatizados

- **Ferramentas Principais (UI):**

- Selenium WebDriver: A ferramenta mais tradicional e poderosa para automação de browsers. Suporta múltiplas linguagens (Java, C#, Python, JavaScript) e browsers.
- Cypress: Framework moderno para E2E que executa no mesmo loop do browser, oferecendo uma experiência de desenvolvimento superior, com time-travel debugging e esperas automáticas. Muito popular para aplicações web.
- Playwright: Sucessor espiritual do Puppeteer, desenvolvido pela Microsoft. Suporta vários browsers (Chromium, Firefox, WebKit) e oferece APIs poderosas para automatizar cenários complexos. Muito rápido e confiável.
- Appium: O equivalente ao Selenium para aplicativos móveis (iOS e Android).

Testes Automatizados

- **Testes de Performance e Carga (Performance & Load Tests)**
 - O que são: Avaliam a velocidade, capacidade de resposta, estabilidade e confiabilidade do software sob uma carga específica (número de usuários simultâneos, volume de dados).
 - Objetivo: Identificar gargalos, garantir que a aplicação atende aos requisitos de performance e descobrir o ponto de quebra do sistema.

Testes Automatizados

- Ferramentas Principais:
 - JMeter: Ferramenta open-source líder de mercado. Projetado para testes de carga, mas muito versátil para vários tipos de teste de performance.
 - LoadRunner, uma das ferramentas mais poderosas e tradicionais do mundo para testes de performance e carga.
 - k6: Ferramenta moderna, de código aberto e com foco em desenvolvedores.
 - Os testes são escritos em JavaScript, é muito performático e é ideal para integrar em pipelines de CI/CD.
 - Gatling: Alto desempenho. Muito eficiente em termos de recursos.

Testes Automatizados

- **Testes de Segurança (Security Tests)**
 - O que são: Procuram vulnerabilidades, falhas e riscos no software que poderiam permitir um ataque malicioso.
 - Objetivo: Proteger dados e garantir que o sistema não seja vulnerável a ameaças conhecidas.

Testes Automatizados

- **Ferramentas Principais:**

- OWASP ZAP (Zed Attack Proxy): Ferramenta open-source líder para testes de segurança em aplicações web. Pode ser usado de forma automática e também por pentesters manuais.
- Burp Suite é uma plataforma integrada usada por profissionais de segurança para realizar testes de segurança em aplicações web
- SonarQube: Plataforma de inspeção contínua de código que detecta bugs, vulnerabilidades e code smells (incluindo problemas de segurança) no código-fonte.
- Snyk / Checkmarx: Ferramentas comerciais especializadas em encontrar e corrigir vulnerabilidades em dependências (SCA - Software Composition Analysis) e no código próprio.

Testes Automatizados

- **Testes de Regressão (Regression Tests)**
 - Não é uma técnica, mas uma prática.
 - Consiste em re-executar suites de testes existentes (de todos os níveis) após uma mudança no código para garantir que novas funcionalidades não quebraram funcionalidades existentes.
- Objetivo: Garantir a estabilidade contínua do produto.
- Ferramentas: Qualquer ferramenta de automação de testes, combinada com um servidor de CI/CD (como Jenkins, GitLab CI, GitHub Actions, Azure DevOps) para automatizar a execução desses testes.

Testes Automatizados

- Testes de unidade são implementados usando-se **frameworks** construídos especificamente para esse fim.
- JUnit.
 - A primeira versão do JUnit foi implementada em conjunto por Kent Beck e Erich Gamma, em 1997, durante uma viagem de avião entre a Suíça e os EUA.

Testes Automatizados

- JUnit
 - é um framework de testes unitários para a linguagem Java.
 - É amplamente utilizado para escrever e executar testes repetíveis de forma automática.
 - Funcionalidades:
 - Anotações: Simplifica a criação de testes com anotações como @Test, @Before e @After.

Testes Automatizados

- Integração com IDEs:
- Integrado com muitas IDEs como Eclipse e IntelliJ, facilitando a execução e o gerenciamento de testes.
- Assertivas: Oferece várias assertivas para verificar resultados esperados.

Testes Automatizados

Exemplo: Teste unitário

```
public class Calculadora {  
    public int soma(int valor1, int valor2){  
        return valor1 + valor2;  
    }  
}
```

- Exemplo:

```
import static org.junit.Assert.*;
```

```
import org.junit.Test;
```

```
public class CalculadoraTest {
```

```
    @Test
```

```
    public void testSoma() {
```

```
        Calculadora calc = new Calculadora();
```

```
        int resultado = calc.soma(10, 5);
```

```
        assertEquals(15, resultado);
```

```
    }
```

```
}
```

Testes

- **JUnit oferece diversas variações de assert,**
- todas têm o mesmo objetivo:
 - testar se um determinado resultado é igual a um valor esperado.

assertTrue verifica se o valor passado como parâmetro é verdadeiro.

assertEquals verifica se o valor dos parametros são iguais

Testes

- Métodos da classe **Assert**
 - Os métodos da classe Assert são os seguintes:
 - void **assertEquals**(valor esperado, valor real):
verifica se dois primitivos/objetos são iguais
(método sobrecarregado).
 - void **assertTrue** (condição booleana): verifica se uma condição é verdadeira.

Testes

- void **assertFalse**(condição booleana): verifica se uma condição é falsa.
- void **assertNull**(Object obj): verifica se o objeto é nulo.
- void **assertNotNull**(Object obj): verifica se o objeto não é nulo.

Testes

- void **assertSame**(object1, object2) - testa se duas referências de objeto apontam para o mesmo objeto.
- void **assertNotSame**(object1, object2) - testa se duas referências de objeto não apontam para o mesmo objeto.
- void **assertArrayEquals**(expectedArray, resultArray)- testa se dois arrays são iguais entre si.

//Verifica se dois objetos são iguais

```
assertEquals(str1, str2);
```

//Verifique se uma condição é verdadeira

```
assertTrue (val1 < val2);
```

//Verifique se uma condição é verdadeira

```
assertFalse(val1 > val2);
```

//Verifique se um objeto não é nulo

```
assertNotNull(str1);
```

//Verifique se um objeto é nulo

assertNull(str3);

//Verifique se duas referências apontam para o mesmo objeto

assertSame(str4,str5);

//Verifique se duas referências não apontam para o mesmo objeto

assertNotSame(str1,str3);

//Verifique se dois arrays são iguais entre si.

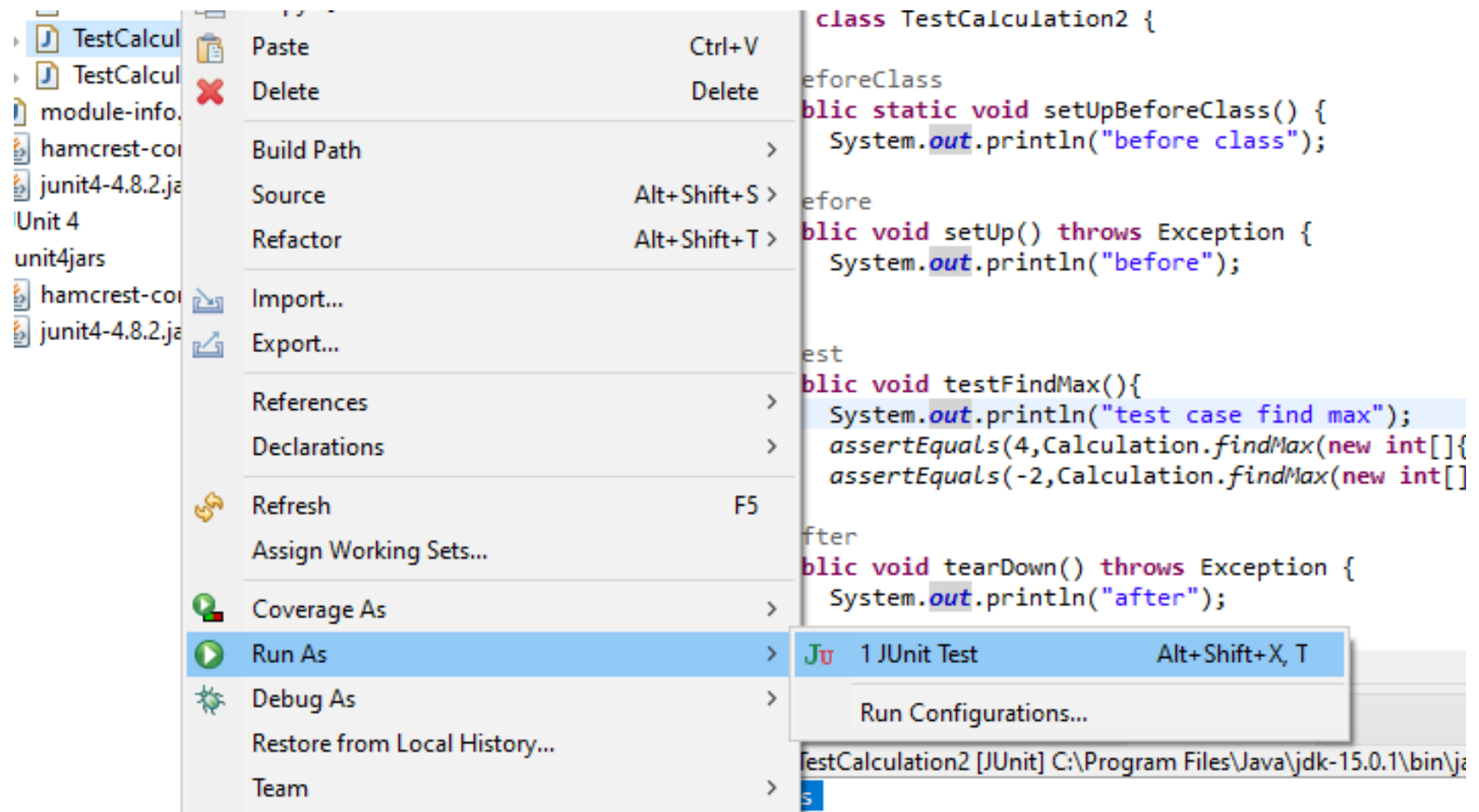
assertArrayEquals(expectedArray, resultArray);

}

}

Testes

- IDEs oferecem suporte para rodar os testes de um sistema (Exemplo - Eclipse),



Testes

- O resultado é mostrado na própria IDE e a barra verde informa que todos os testes passaram.
- Pode-se observar também que o teste roda rapidamente, em 0.025 segundos.

Finished after 0.025 seconds

Runs: 1/1

✖ Errors: 0

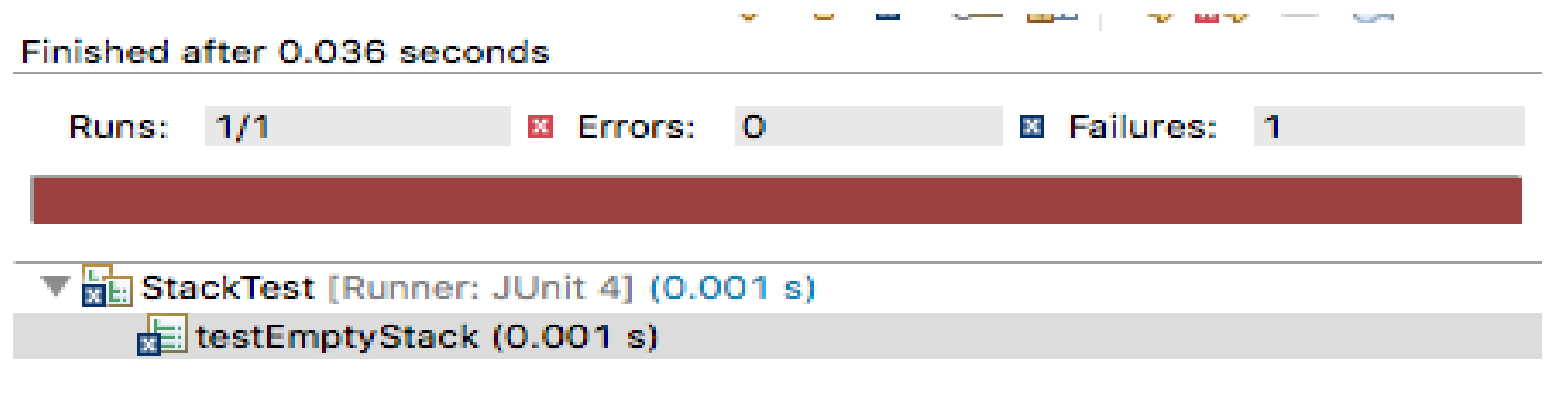
✖ Failures: 0



▶  StackTest [Runner: JUnit 4] (0.000 s)

Testes

- Suponha que tivéssemos cometido um erro na implementação da classe.
 - **Por exemplo, suponha que o atributo size fosse inicializado com o valor 1**, em vez de zero. Nesse caso, a execução dos testes iria falhar, como mostrado pela barra vermelha na IDE:



Testes

- A mensagem de erro informa que houve uma falha durante a execução de `testEmptyStack`.
- **Falha (failure) é o termo usado pelo JUnit** para indicar testes cujo comando `assert` não foi satisfeito.
 - Em uma outra janela da IDE, pode-se descobrir que a asserção responsável pela falha encontra-se na linha 19 do arquivo `StackTest.java`.

 Failure Trace

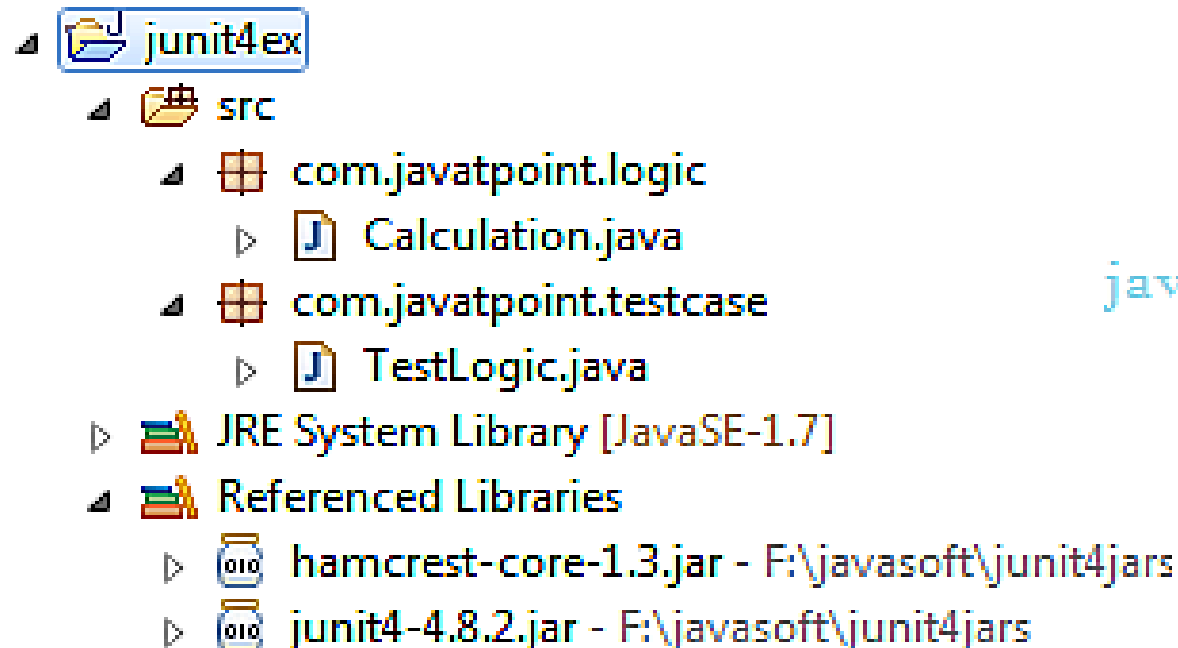


 `java.lang.AssertionError`

 `at StackTest.testEmptyStack(StackTest.java:19)`

Testes

- Arquivos jar necessários - JUnit
- Você precisa carregar os arquivos junit4.jar e hamcrest-core.jar.
 - <https://junit.org/junit4/>
- Exemplo (Eclipse):



Testes

- Para configurar o JUnit com Eclipse, siga as etapas abaixo.
- https://www.tutorialspoint.com/junit/junit_plug_with_eclipse.htm

Testes

- Dúvidas?